

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS2DA: Data Structures & Algorithms

Introduction

Algorithms — in common usage

Algorithm is often used to describe when a computer makes an unaccountable or unexplainable decision.

<https://www.bbc.co.uk/news/technology-14306146> (2011)

In reality, our electronic overlords are already taking control, and they are doing it in a far more subtle way than science fiction would have us believe.

Their weapon of choice — the algorithm.

Algorithms — in common usage

Algorithm is often used to describe when a computer makes an unaccountable or unexplainable decision.

<https://www.bbc.co.uk/bitesize/articles/zd9tt39>

*The decisions around the types of posts that we see when scrolling through social media are in the hands of **algorithms** rather than people, computer programs **ran** by companies that own the app or website that filter the posts in your feed.*

...if a news story comes out that might not be in line with your opinions and beliefs, the algorithms might not surface it in your feed.

Algorithms — according to Computer Science

Indeed, computers make decisions using algorithms, but the real meaning is broader and less sinister.

An algorithm is a well-defined sequence of steps for solving a problem.

We need to understand algorithms to make sure programs we write give the right answer in a reasonable amount of time.

Mostly, software developers use existing algorithms rather than create new ones. Even so, understanding algorithms helps with selecting the best one to use.

Properties of Algorithms

Donald Knuth says that algorithms have:

- **Finiteness** — guaranteed to terminate
- **Definiteness** — unambiguously described steps
- **Input** — zero or more
- **Output** — one or more
- **Effectiveness** — steps simple enough for a computer or human clerk to follow

We can also add:

- **Correctness** — guaranteed to give the right answer

Data Structures

All algorithms operate on data.

Niklaus Wirth: **Algorithms + Data Structures = Programs**

The data must be stored somehow in the computer's memory.

Often, there is more than one way of organising the data.

How the data is stored affects the **efficiency** of different operations — and hence algorithms using the data structure.

Efficiency typically means speed (**time**) or memory usage (**space**).

Lists

Suppose we want to store a **list** of values.

We want to be able to add new values, look up values, remove existing values...

How do we store it in memory?

Most obvious answer: **Array**

Second most obvious answer: **Linked list**

Exercise: What are the advantages/disadvantages?

Abstract Data Types

Java's `ArrayList` and `LinkedList` are concrete data structures.

We know (or can find out) exactly how they are stored in memory and which algorithms they use.

But they both implement the same "List" operations and behaviour.

In a program, we can swap an `ArrayList` for a `LinkedList` and the program will mostly behave the same — but perhaps faster/slower.

We can say that List is an **abstract data type** (ADT).

Important ADTs in Java's Collections Framework: **Set**, **List**, **Deque**, **Map**.

Today: Hash Tables and Quicksort

Two common operations on data structures are **searching** and **sorting**.

Hash tables are a common data structure that allows for fast searching.

Quicksort is a common algorithm for fast sorting of an array.

Neither is theoretically optimal, but they are both extremely fast in most cases.

Maps

A **Map** stores **keys** and associates them with **values**.

Suppose we want to record how many days there are in each month of the year.

We could use an array:

```
int months[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

Key — index of month

Value — number of days in month

For example, in April (4th month of year), there are `months[4-1]` days.

The key 4 is **mapped** to the **value** 30. Fine if keys are sequential integers.

What if we want to use strings, such as the names of the months?

Lists as Maps

A **Map** needs to support the following operations:

- `insert(k,v)` — add the **mapping** $k \rightarrow v$ to the map
- `get(k)` — get the value v associated with k (if any)
- `remove(k)` — remove the key k and its associated v

If we don't care about efficiency, we store the map as a List of pairs.

Exercise: How fast are the operations?

Hash Tables

Hash tables use **arrays** to give a map with fast operations.

Idea

- Write a function `hash()`, such that `hash(k)` is a small integer called a **hash code**.
- Make an array of pairs `table[]` the size of the maximum possible hash code.
- Store the pair `(k, v)` in "**bucket**" at array index `hash(k)`.

Operations

- `insert(k,v)` — `table[hash(k)] = {k, v}`
- `get(k)` — if `table[hash(k)] = {k, v}` then return `v` else return nothing
- `remove(k)` — if `table[hash(k)] = {k, _}` then `table[hash(k)] = empty`

Hash Functions

Problem 1: How do we write `hash()` ? It depends on the type of keys.

Integer keys — take modulus (`%`) with prime number `M`

String keys — Sedgewick suggests:

```
int hash = 0;
for (int i = 0; i < s.length(); i++)
    hash = (R * hash + s.charAt(i)) % M;
```

where `R` is a small prime such as 31.

We can extend the string technique to blocks of data of any size.

Worked Example — Days in each Month

With $R = 31$ and $M = 15$:

Name	Hash	Days	—	Name	Hash	Days
January	10	31		July	0	31
February	7	28		August	3	31
March	11	31		September	5	30
April	9	30		October	13	31
May	10	31		November	5	30
June	12	30		December	11	31

Problem 2: What if two distinct keys have the same hash, so **collide** in the same bucket?

Hash Tables

Hash tables use **arrays** and **linked lists** to give a map with fast operations.

Idea

- Write a function `hash()`, such that `hash(k)` is a small integer called a **hash code**.
- Make an array of pairs `table[]` the size of the maximum possible hash code.
- Store a **linked list** of `(k, v)` pairs at array index `hash(k)`.

Operations

- `insert(k,v)` — `table[hash(k)].insert(k, v)`
- `get(k)` — return `table[hash(k)].get(k)`
- `remove(k)` — `table[hash(k)].remove(k)`

Worked Example — Days in each Month

With $R = 31$ and $M = 15$:

Name	Hash	Days	—	Name	Hash	Days
January	10	31		July	0	31
February	7	28		August	3	31
March	11	31		September	5	30
April	9	30		October	13	31
May	10	31		November	5	30
June	12	30		December	11	31

Hash Tables — The Ideal Case and Worst Case

Exercise: When is the performance of hash tables as good as possible?

Exercise: When is the performance of hash tables as bad as possible?

Hash Tables — The Average Case

With "good" choice of hash function, all operations in a hash table are "typically" **constant time**: they don't depend on the size of the data structure.

Library implementations typically monitor how full the table is and resize it if most of the buckets are used. This requires recalculation of all hash values.

Security risk: An attacker who knows the implementation could feed in deliberately bad data to degrade program performance.

Mitigation: Use a balanced binary tree (see later lecture) instead of the linked list, or instead of the hash table.

Despite subtleties, hash tables are so fast and convenient that they are used almost everywhere.

Compared with just using a giant array, hash tables **trade time for space**.

Aside: Cryptographic Hash Functions

In cryptography, hash functions are used in digital signatures and for checking passwords without storing them.

Cryptographic hashes must be very carefully designed.

It should be computationally very hard to:

- find two inputs (keys) with the same hash;
- work out a single bit of the input from the hash.

Modern cryptographic hashes often use a complex combination of additions, rotations (bit shifts) and XORs.

What does algorithm P do?

inputs: A: array; m: value in A
output: modified A

```
i ← 0
j ← 0
k ← size of A - 1
while j ≤ k:
    if A[j] < m:
        swap A[i] and A[j]
        i ← i + 1
        j ← j + 1
    else if A[j] > m:
        swap A[j] and A[k]
        k ← k - 1
    else:
        j ← j + 1
```

Note: ← means assignment (=).

Pseudo-code

An algorithm is the "same" algorithm whether it is written in C, Java, Python or English...
...provided we don't compromise **definiteness** and **effectiveness**.

Algorithms can be specified in **pseudo-code** — something that looks a bit like code in a real programming language, but may include natural language (English).

There isn't a standardised pseudo-code language. Some textbooks are stricter than others about how much English is allowed.

Pseudo-code makes it easier for a human to grasp the ideas of the algorithm without worrying about programming language details.

Module Information

CS2DA: Data Structures & Algorithms

Timetabled teaching

- 11 x 2-hour lectures — Weeks 1-5, 7-12
 - Weeks 1-5, 12 — Martin Lester
 - Weeks 7-11, 12 — Xia Hong
- 3 x 2-hour tutorials — Weeks 1,3,5
- 7 x 2-hour practicals — Weeks 2,4,7-11

Assessment

- Technical report — 50% — due Week 11
- Exam — 50% — Assessment Period

CS2DA: Data Structures & Algorithms

Textbooks

- Algorithms, 4th Edition — **Sedgewick** and Wayne
- Introduction to Algorithms, 4th Edition — **Cormen, Leiserson, Rivest, Stein**
- Fundamentals of Computer Algorithms — **Horowitz** and Sahni

Blackboard

- Lecture slides
- Practical exercises
- Tutorial exercises
- Recommended reading

Quicksort

Sorting

One of the most common operations on a list is **sorting**.

Algorithms are often faster on sorted data.

Examples:

- Sorted lists can be searched efficiently using **binary search**.
- We can immediately find the **median** of a sorted list.

Recall: Bubble Sort

One of the most basic sorting algorithms is **bubble sort**.

Idea: If two adjacent elements in an array are the wrong way round, swap them. Eventually, everything will be in order.

Exercise: Write code for bubble sort.

Bubble sort is fast for an almost sorted list, but can be very slow.

Quicksort

```
To sort an array A:  
  if size of A <= 1 then return  
  pick a pivot p from A  
  partition A into 3 sections: less than p, equal to p, greater than p  
  sort section less than p  
  sort section greater than p
```

How does this ensure A is sorted?

Is this an algorithm?

Quicksort

How to partition?

Use algorithm P — the Dutch National Flag algorithm.

How to pick a pivot?

- Pick first element in array?
- Pick random element in array?
- Cleverly choose "best" option?

Worked Example

Let's sort: 7,3,9,8,1,5,4,0

Use first element in array as pivot.

The Ideal Case

Most of the work is in partitioning — time proportional to length of array.

Each time we split recursively, the two subproblems are smaller than the original array.

Running time is roughly proportional to length of array times depth of recursion.

We want to minimise depth of recursion.

The subproblems shrink most if the pivot ends up in the middle.

The median element is the ideal pivot. But finding it is slow in practice.

The Worst Case

Conversely, the worst case is when one subproblem is almost empty and the other is almost the whole array.

The worst case occurs when the pivot ends up at either end.

Exercise: If we pick the pivot to be the first element, construct an input array that gives worst case performance.

Again, this is a potential security vulnerability if an attacker knows the algorithm used and can control the input.

The Average Case?

In practice, with **sensible pivot choice**, the worst case **rarely** happens.

With random pivot choice, 50% chance of reducing size by at least 25%.

Double depth of ideal case is still very good.

Lester's Linear Slowdown Slogan

The algorithm that takes twice as long is just as fast.

Formal explanation in next lecture.

Divide & Conquer

Quicksort is an example of a **divide & conquer** algorithm.

We **divide** the problem of **sorting one big array** into smaller subproblems that we can **conquer: sorting two smaller arrays**.

We do this **recursively** until we get a tiny **base case** problem that we can solve immediately: **sorting an empty or single element array**.

Then we **combine** the results of the subproblems to get the solution to the whole problem. (In the case of quicksort, because the subarrays are sorted in place, there is nothing to do.)

Recap

An algorithm is a well-defined sequence of steps for solving a problem.

Important properties of algorithms: finiteness, definiteness, input, output, effectiveness, correctness.

Algorithms + Data Structures = Programs

Hash tables use arrays and linked lists to give a map with fast operations.

Quicksort uses divide & conquer to sort an array quickly.

Hash tables and quicksort are: ubiquitous, conceptually simple, fiendishly subtle, fast in the typical case, slow in the worst case, potential security vulnerabilities.

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Next time: Divide & Conquer